Self-Supervised Learning for Named Entity Recognition (NER)

Objective

This project implements a **Self-Supervised Learning framework** for **Named Entity Recognition (NER)** using a Transformer-based architecture (DistilBERT).

The training pipeline consists of two stages:

1. **Self-Supervised Pretraining (Masked Language Modeling - MLM)**
2. **Supervised Fine-Tuning for NER**

The goal is to leverage large-scale unlabeled text to learn strong contextual representations before adapting the model to a downstream token classification task.

Project Overview

Modern NLP systems use self-supervised learning to pretrain language models on massive unlabeled corpora. These pretrained models are then fine-tuned on specific tasks like NER.

In this project:

- WikiText-2 is used for **MLM pretraining**
- CoNLL-2003 is used for **NER fine-tuning**
- DistilBERT serves as the backbone Transformer model

Stage 1: Self-Supervised Pretraining (MLM)

Dataset

- **WikiText-2 (raw version)**
- Unlabeled text corpus

Pretext Task

- **Masked Language Modeling (MLM)**
- Randomly masks 15% of tokens
- Model predicts masked tokens using context

Model

- `DistilBertForMaskedLM`

- Pretrained `distilbert-base-uncased`
- Fine-tuned for 1 epoch

Outcome

The model learns:

- Contextual word representations
- Semantic relationships
- Long-range dependencies

Stage 2: Supervised NER Fine-Tuning

Dataset

- **CoNLL-2003**
- Standard benchmark for NER

Entity Classes

- **PER** – Person
- **ORG** – Organization
- **LOC** – Location
- **MISC** – Miscellaneous

Model

- `DistilBertForTokenClassification`
- Loads encoder weights from MLM phase
- Adds classification head

Training

- 2 epochs
- F1 score used for evaluation

Results

Metric	Value
Validation F1	**0.93**
Validation Loss	0.058
Precision (avg)	0.93
Recall (avg)	0.94

Per-Entity Performance

- PER → 0.97 F1
- LOC → 0.95 F1

- ORG → 0.90 F1
- MISC → 0.83 F1

The model demonstrates strong contextual understanding and accurate entity classification.

Example Inference

Input:

```
A person was born in Hyderabad and worked at Microsoft.
```

Predicted Entities:

- Hyderabad → LOC
- Microsoft → ORG

Architecture Summary

```
Unlabeled Text (WikiText-2)
↓
Masked Language Modeling (MLM)
↓
Pretrained DistilBERT Encoder
↓
Add Token Classification Head
↓
Fine-Tune on CoNLL-2003
↓
Named Entity Recognition
```

Deliverables

- Data preprocessing pipeline
- Self-supervised MLM pretraining
- Transformer-based architecture
- NER fine-tuning
- F1 evaluation & classification report
- Inference pipeline
- Saved trained model

Conclusion

This project successfully demonstrates:

- Self-supervised representation learning
- Transfer learning using Transformers
- Token-level classification for NER
- End-to-end training and deployment

The achieved F1 score (~93%) confirms the effectiveness of combining self-supervised pretraining with supervised fine-tuning.

Technologies Used

- PyTorch
- HuggingFace Transformers
- HuggingFace Datasets
- SeqEval
- Google Colab (GPU)

Code:

```
# ============================================================
# SELF-SUPERVISED LEARNING FOR NER
# (MLM PRE-TRAINING + NER FINE-TUNING)
# ============================================================

!pip install -U transformers datasets accelerate seqeval

import torch
import numpy as np
from datasets import load_dataset
from transformers import (
    DistilBertTokenizerFast,
    DistilBertForMaskedLM,
    DistilBertForTokenClassification,
    DataCollatorForLanguageModeling,
    Trainer,
    TrainingArguments,
    pipeline
)
from seqeval.metrics import f1_score, classification_report

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# ------------------------------------------------------------
# 1. Load Unlabeled Dataset (Self-Supervised Phase - MLM)
```

```
# ------------------------------------------------------------

print("\nLoading WikiText-2 Dataset for MLM...\n")
mlm_dataset = load_dataset("wikitext", "wikitext-2-raw-v1")
print(mlm_dataset)

# ------------------------------------------------------------
# 2. Tokenization for MLM
# ------------------------------------------------------------

tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-
uncased")

def tokenize_mlm(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        padding="max_length",
        max_length=128
    )

tokenized_mlm = mlm_dataset.map(
    tokenize_mlm,
    batched=True,
    remove_columns=["text"]
)

tokenized_mlm.set_format("torch")
print("\nMLM Dataset Tokenized Successfully.\n")

# ------------------------------------------------------------
# 3. Define Pretext Task (Masked Language Modeling)
# ------------------------------------------------------------

mlm_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=True,
    mlm_probability=0.15
)

# ------------------------------------------------------------
# 4. Initialize Transformer Model (DistilBERT)
# ------------------------------------------------------------

mlm_model = DistilBertForMaskedLM.from_pretrained(
    "distilbert-base-uncased"
)
mlm_model.to(device)

# ------------------------------------------------------------
```

```python
# 5. Self-Supervised Pre-Training (MLM)
# ------------------------------------------------------------

mlm_training_args = TrainingArguments(
    output_dir="./distilbert-mlm-pretrained",
    learning_rate=5e-5,
    weight_decay=0.01,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=1,
    logging_steps=100,
    eval_strategy="no",
    fp16=torch.cuda.is_available(),
)

mlm_trainer = Trainer(
    model=mlm_model,
    args=mlm_training_args,
    train_dataset=tokenized_mlm["train"],
    eval_dataset=tokenized_mlm["validation"],
    data_collator=mlm_collator,
)

print("\nStarting Self-Supervised Pre-training...\n")
mlm_trainer.train()

print("\nSaving Pre-trained Model...\n")
mlm_model.save_pretrained("./distilbert-mlm-pretrained")
tokenizer.save_pretrained("./distilbert-mlm-pretrained")

# ============================================================
# SUPERVISED PHASE: NER FINE-TUNING
# ============================================================

# ------------------------------------------------------------
# 6. Load Labeled Dataset (CoNLL-2003)  [2026 SAFE VERSION]
# ------------------------------------------------------------

print("\nDownloading Official CoNLL-2003 Dataset...\n")

!wget -q https://data.deepai.org/conll2003.zip

import zipfile
with zipfile.ZipFile("conll2003.zip", "r") as zip_ref:
    zip_ref.extractall("conll2003_data")

from datasets import Dataset, DatasetDict

def read_conll_file(filepath):
    tokens = []
```

```
    ner_tags = []
    current_tokens = []
    current_tags = []

    with open(filepath, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line == "":
                if current_tokens:
                    tokens.append(current_tokens)
                    ner_tags.append(current_tags)
                    current_tokens = []
                    current_tags = []
            else:
                splits = line.split()
                if len(splits) >= 4:
                    current_tokens.append(splits[0])
                    current_tags.append(splits[-1])

    return {"tokens": tokens, "ner_tags": ner_tags}

train_data = read_conll_file("conll2003_data/train.txt")
valid_data = read_conll_file("conll2003_data/valid.txt")
test_data  = read_conll_file("conll2003_data/test.txt")

ner_dataset = DatasetDict({
    "train": Dataset.from_dict(train_data),
    "validation": Dataset.from_dict(valid_data),
    "test": Dataset.from_dict(test_data),
})

print(ner_dataset)

# Build label list
label_list = sorted(list(set(tag for tags in train_data["ner_tags"]
for tag in tags)))
label2id = {label: i for i, label in enumerate(label_list)}
id2label = {i: label for label, i in label2id.items()}
num_labels = len(label_list)

# Encode labels
def encode_labels(example):
    example["ner_tags"] = [label2id[tag] for tag in
example["ner_tags"]]
    return example

ner_dataset = ner_dataset.map(encode_labels)

# ------------------------------------------------------------
# 7. Tokenization & Label Alignment for NER
```

```
# ------------------------------------------------------------

def tokenize_ner(examples):
    tokenized_inputs = tokenizer(
        examples["tokens"],
        truncation=True,
        padding="max_length",
        max_length=128,
        is_split_into_words=True
    )

    labels = []
    for i, label in enumerate(examples["ner_tags"]):
        word_ids = tokenized_inputs.word_ids(batch_index=i)
        label_ids = []
        previous_word_idx = None

        for word_idx in word_ids:
            if word_idx is None:
                label_ids.append(-100)
            elif word_idx != previous_word_idx:
                label_ids.append(label[word_idx])
            else:
                label_ids.append(label[word_idx])
            previous_word_idx = word_idx

        labels.append(label_ids)

    tokenized_inputs["labels"] = labels
    return tokenized_inputs

tokenized_ner = ner_dataset.map(tokenize_ner, batched=True)
tokenized_ner.set_format("torch")

print("\nNER Dataset Tokenized Successfully.\n")

# ------------------------------------------------------------
# 8. Load Pre-trained Encoder for NER
# ------------------------------------------------------------

ner_model = DistilBertForTokenClassification.from_pretrained(
    "./distilbert-mlm-pretrained",
    num_labels=num_labels,
    id2label=id2label,
    label2id=label2id
)

ner_model.to(device)

# ------------------------------------------------------------
```

```python
# 9. Fine-Tune for NER (Supervised Training)
# ------------------------------------------------------------

def compute_metrics(p):
    predictions, labels = p
    predictions = np.argmax(predictions, axis=2)

    true_labels = [
        [id2label[l] for l in label if l != -100]
        for label in labels
    ]
    true_predictions = [
        [id2label[p] for (p, l) in zip(pred, label) if l != -100]
        for pred, label in zip(predictions, labels)
    ]

    return {"f1": f1_score(true_labels, true_predictions)}

ner_training_args = TrainingArguments(
    output_dir="./distilbert-ner-model",
    learning_rate=3e-5,
    weight_decay=0.01,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    logging_steps=100,
    eval_strategy="epoch",
    fp16=torch.cuda.is_available(),
)

ner_trainer = Trainer(
    model=ner_model,
    args=ner_training_args,
    train_dataset=tokenized_ner["train"],
    eval_dataset=tokenized_ner["validation"],
    compute_metrics=compute_metrics,
)

print("\nStarting NER Fine-Tuning...\n")
ner_trainer.train()

print("\nEvaluating NER Model...\n")
eval_results = ner_trainer.evaluate()
print("\nNER Evaluation Results:\n", eval_results)

# ------------------------------------------------------------
# 10. Detailed Classification Report
# ------------------------------------------------------------

predictions, labels, _ =
```

```
ner_trainer.predict(tokenized_ner["validation"])
preds = np.argmax(predictions, axis=2)

true_labels = [
    [label_list[l] for l in label if l != -100]
    for label in labels
]
true_predictions = [
    [label_list[p] for (p, l) in zip(pred, label) if l != -100]
    for pred, label in zip(preds, labels)
]

print("\nClassification Report:\n")
print(classification_report(true_labels, true_predictions))

# ------------------------------------------------------------
# 11. Inference Pipeline (Deployment Ready)
# ------------------------------------------------------------

print("\nRunning NER Inference...\n")

ner_pipeline = pipeline(
    "token-classification",
    model=ner_model,
    tokenizer=tokenizer,
    aggregation_strategy="simple"
)

test_sentence = "A person was born in Hyderabad and worked at
Microsoft."
results = ner_pipeline(test_sentence)

print("Input:", test_sentence)
print("\nPredicted Entities:")
for r in results:
    print(f"Entity: {r['word']}, Label: {r['entity_group']},
Confidence: {round(r['score'],4)}")

# ------------------------------------------------------------
# 12. Save Final Model
# ------------------------------------------------------------

ner_model.save_pretrained("./final_ner_model")
tokenizer.save_pretrained("./final_ner_model")

# ------------------------------------------------------------
# Deliverables Summary
# ------------------------------------------------------------

print("\n" + "="*60)
```

```
print("SELF-SUPERVISED NER PROJECT - DELIVERABLES REPORT")
print("="*55)
print("(a) Data & Preprocessing Pipeline: COMPLETED")
print("(b) Self-Supervised Model (MLM): PRE-TRAINED")
print("(c) NER Fine-Tuned Model: TRAINED & SAVED")
print("(d) Evaluation: F1 Score + Classification Report Generated")
print("(e) Deployment: Inference Pipeline Ready")
print("="*55)
```

```
Requirement already satisfied: transformers in
/usr/local/lib/python3.12/dist-packages (5.0.0)
Collecting transformers
  Downloading transformers-5.2.0-py3-none-any.whl.metadata (32 kB)
Requirement already satisfied: datasets in
/usr/local/lib/python3.12/dist-packages (4.0.0)
Collecting datasets
  Downloading datasets-4.5.0-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: accelerate in
/usr/local/lib/python3.12/dist-packages (1.12.0)
Collecting seqeval
  Downloading seqeval-1.2.2.tar.gz (43 kB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.6/43.6 kB 2.3 MB/s eta
0:00:00
etadata (setup.py) ... ent already satisfied: huggingface-
hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from
transformers) (1.4.1)
Requirement already satisfied: numpy>=1.17 in
/usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in
/usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in
/usr/local/lib/python3.12/dist-packages (from transformers)
(2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.22.2)
Requirement already satisfied: typer-slim in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in
/usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from datasets) (3.24.2)
Collecting pyarrow>=21.0.0 (from datasets)
  Downloading pyarrow-23.0.1-cp312-cp312-
manylinux_2_28_x86_64.whl.metadata (3.1 kB)
Requirement already satisfied: dill<0.4.1,>=0.3.0 in
/usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
```

```
Requirement already satisfied: pandas in
/usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in
/usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: httpx<1.0.0 in
/usr/local/lib/python3.12/dist-packages (from datasets) (0.28.1)
Requirement already satisfied: xxhash in
/usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.19 in
/usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.10.0,>=2023.1.0 in
/usr/local/lib/python3.12/dist-packages (from
fsspec[http]<=2025.10.0,>=2023.1.0->datasets) (2025.3.0)
Requirement already satisfied: psutil in
/usr/local/lib/python3.12/dist-packages (from accelerate) (5.9.5)
Requirement already satisfied: torch>=2.0.0 in
/usr/local/lib/python3.12/dist-packages (from accelerate)
(2.10.0+cu128)
Requirement already satisfied: scikit-learn>=0.21.3 in
/usr/local/lib/python3.12/dist-packages (from seqeval) (1.6.1)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in
/usr/local/lib/python3.12/dist-packages (from
fsspec[http]<=2025.10.0,>=2023.1.0->datasets) (3.13.3)
Requirement already satisfied: anyio in
/usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets)
(4.12.1)
Requirement already satisfied: certifi in
/usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets)
(2026.1.4)
Requirement already satisfied: httpcore==1.* in
/usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets)
(1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-
packages (from httpx<1.0.0->datasets) (3.11)
Requirement already satisfied: h11>=0.16 in
/usr/local/lib/python3.12/dist-packages (from httpcore==1.*-
>httpx<1.0.0->datasets) (0.16.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=1.3.0->transformers) (1.2.0)
Requirement already satisfied: shellingham in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=1.3.0->transformers) (1.5.4)
Requirement already satisfied: typing-extensions>=4.1.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=1.3.0->transformers) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.32.2-
>datasets) (3.4.4)
```

```
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.32.2-
>datasets) (2.5.0)
Requirement already satisfied: scipy>=1.6.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.21.3-
>seqeval) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.21.3-
>seqeval) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.21.3-
>seqeval) (3.6.0)
Requirement already satisfied: setuptools in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (3.6.1)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (3.1.6)
Requirement already satisfied: cuda-bindings==12.9.4 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
```

```
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>accelerate) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4-
>torch>=2.0.0->accelerate) (1.3.4)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas->datasets)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas->datasets)
(2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas->datasets)
(2025.3)
Requirement already satisfied: typer>=0.24.0 in
/usr/local/lib/python3.12/dist-packages (from typer-slim-
>transformers) (0.24.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.10.0,>=2023.1.0->datasets) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.10.0,>=2023.1.0->datasets) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
```

```
=4.0.0a1->fsspec[http]<=2025.10.0,>=2023.1.0->datasets) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.10.0,>=2023.1.0->datasets) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.10.0,>=2023.1.0->datasets) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.10.0,>=2023.1.0->datasets) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.10.0,>=2023.1.0->datasets) (1.22.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2-
>pandas->datasets) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3-
>torch>=2.0.0->accelerate) (1.3.0)
Requirement already satisfied: click>=8.2.1 in
/usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-
slim->transformers) (8.3.1)
Requirement already satisfied: rich>=12.3.0 in
/usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-
slim->transformers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in
/usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-
slim->transformers) (0.0.4)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch>=2.0.0-
>accelerate) (3.0.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=12.3.0-
>typer>=0.24.0->typer-slim->transformers) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=12.3.0-
>typer>=0.24.0->typer-slim->transformers) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0-
>rich>=12.3.0->typer>=0.24.0->typer-slim->transformers) (0.1.2)
Downloading transformers-5.2.0-py3-none-any.whl (10.4 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.4/10.4 MB 87.9 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 515.2/515.2 kB 29.5 MB/s eta
0:00:00
anylinux_2_28_x86_64.whl (47.6 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 47.6/47.6 MB 16.6 MB/s eta
0:00:00
e=seqeval-1.2.2-py3-none-any.whl size=16162
```

```
sha256=47b9ba1daf3e4c06870d7b44d95a80c7d60840288c4e1b9bbeb5082497276fb
0
  Stored in directory:
/root/.cache/pip/wheels/5f/b8/73/0b2c1a76b701a677653dd79ece07cfabd7457
989dbfbdcd8d7
Successfully built seqeval
Installing collected packages: pyarrow, seqeval, datasets,
transformers
  Attempting uninstall: pyarrow
    Found existing installation: pyarrow 18.1.0
    Uninstalling pyarrow-18.1.0:
      Successfully uninstalled pyarrow-18.1.0
  Attempting uninstall: datasets
    Found existing installation: datasets 4.0.0
    Uninstalling datasets-4.0.0:
      Successfully uninstalled datasets-4.0.0
  Attempting uninstall: transformers
    Found existing installation: transformers 5.0.0
    Uninstalling transformers-5.0.0:
      Successfully uninstalled transformers-5.0.0
Successfully installed datasets-4.5.0 pyarrow-23.0.1 seqeval-1.2.2
transformers-5.2.0
Using device: cuda

Loading WikiText-2 Dataset for MLM...


/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(

{"model_id":"4305297cfcc640beb25634e9ff75c7c9","version_major":2,"vers
ion_minor":0}

{"model_id":"d0ab3b3eaff44ba5acc8ef7fbcdd5077","version_major":2,"vers
ion_minor":0}

{"model_id":"8587f47d728f49769a06706b3408675a","version_major":2,"vers
ion_minor":0}

{"model_id":"0697effcde514d3e80626cda882e1455","version_major":2,"vers
ion_minor":0}
```

{"model_id":"87899733094745f2a7bae4e07f37a874","version_major":2,"version_minor":0}

{"model_id":"0086152987be43529be0d1a1ecffbd10","version_major":2,"version_minor":0}

{"model_id":"da1f05f88b854edf82163e8adb07e181","version_major":2,"version_minor":0}

```
DatasetDict({
    test: Dataset({
        features: ['text'],
        num_rows: 4358
    })
    train: Dataset({
        features: ['text'],
        num_rows: 36718
    })
    validation: Dataset({
        features: ['text'],
        num_rows: 3760
    })
})
```

{"model_id":"5665ac49aa5f42d1901f482dae0da3e6","version_major":2,"version_minor":0}

{"model_id":"3b771bfad2554654b48aed1cdcece973","version_major":2,"version_minor":0}

{"model_id":"34516ec3dec643449311ceaff2b890d3","version_major":2,"version_minor":0}

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```

{"model_id":"8a754394bf3d42ad898bb68b74e31e0c","version_major":2,"version_minor":0}

{"model_id":"77b8d661db804a14b32d0834392df5f0","version_major":2,"version_minor":0}

{"model_id":"775d6aff8d8648c2bf7ddc2bceda093c","version_major":2,"version_minor":0}

```
MLM Dataset Tokenized Successfully.
```

{"model_id":"9408ccb224ad4f2194e21113e3b2a856","version_major":2,"version_minor":0}

{"model_id":"215b68e844114169ad8b168bd966301f","version_major":2,"version_minor":0}

{"model_id":"4b889071217a405aae5d9bd6c9393547","version_major":2,"version_minor":0}

```
Starting Self-Supervised Pre-training...
```

```
<IPython.core.display.HTML object>
```

{"model_id":"5593239548f141d38c99b24f58c0a926","version_major":2,"version_minor":0}

{"model_id":"481994e13702439497b78b5a5672aff1","version_major":2,"version_minor":0}

{"model_id":"cfab7948e71c401c8bbfaec2f8cac227","version_major":2,"version_minor":0}

{"model_id":"d91d56fb81da44738bb9955c31d5e16f","version_major":2,"version_minor":0}

{"model_id":"48271c370c614295adc52230d4bc8e75","version_major":2,"version_minor":0}

```
Saving Pre-trained Model...
```

{"model_id":"a8a3e5d75880424d807fdc053b3e471f","version_major":2,"version_minor":0}

```
Downloading Official CoNLL-2003 Dataset...
```

```
DatasetDict({
    train: Dataset({
        features: ['tokens', 'ner_tags'],
        num_rows: 14987
    })
    validation: Dataset({
        features: ['tokens', 'ner_tags'],
        num_rows: 3466
    })
    test: Dataset({
        features: ['tokens', 'ner_tags'],
        num_rows: 3684
```

```
    })
})
```


```
{"model_id":"91b9516933f34fbfa11a67663908670c","version_major":2,"vers
ion_minor":0}

{"model_id":"10049c23d6584032baa794ce2e93eea5","version_major":2,"vers
ion_minor":0}

{"model_id":"fb1a7548a67f4f168820afef064d4978","version_major":2,"vers
ion_minor":0}

{"model_id":"246ebada7b1742bfa211ff081324a634","version_major":2,"vers
ion_minor":0}

{"model_id":"ebf2bf1133d149af8e18aa3a5743deba","version_major":2,"vers
ion_minor":0}

{"model_id":"c07d59f45c80406386a8413aa0ff885d","version_major":2,"vers
ion_minor":0}
```


```
NER Dataset Tokenized Successfully.
```


```
{"model_id":"29dbd928b5c24cc1b5e842304d73ddc0","version_major":2,"vers
ion_minor":0}
```


```
DistilBertForTokenClassification LOAD REPORT from: ./distilbert-mlm-
pretrained
Key                       | Status     |
--------------------------+------------+-
vocab_projector.bias      | UNEXPECTED |
vocab_layer_norm.weight   | UNEXPECTED |
vocab_transform.weight    | UNEXPECTED |
vocab_transform.bias      | UNEXPECTED |
vocab_layer_norm.bias     | UNEXPECTED |
classifier.bias           | MISSING    |
classifier.weight         | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
- MISSING  :those params were newly initialized because missing from
the checkpoint. Consider training on your downstream task.


Starting NER Fine-Tuning...


<IPython.core.display.HTML object>
```

{"model_id":"24570c052d58427794868a84b7b57995","version_major":2,"version_minor":0}

{"model_id":"4fb239bca41043feb4934c0465a88dea","version_major":2,"version_minor":0}

{"model_id":"dcd670a7f74f452a9024a7311328f2f2","version_major":2,"version_minor":0}

{"model_id":"f866fb27fc5a4edaab4523a02f2723c8","version_major":2,"version_minor":0}

```
Evaluating NER Model...


<IPython.core.display.HTML object>


NER Evaluation Results:
 {'eval_loss': 0.058185599744319916, 'eval_f1': 0.931241655540721,
'eval_runtime': 3.6725, 'eval_samples_per_second': 943.778,
'eval_steps_per_second': 59.088, 'epoch': 2.0}

Classification Report:

              precision    recall  f1-score   support

         LOC       0.95      0.96      0.95      2618
        MISC       0.83      0.83      0.83      1231
         ORG       0.89      0.91      0.90      2056
         PER       0.97      0.98      0.97      3029

   micro avg       0.93      0.94      0.93      8934
   macro avg       0.91      0.92      0.91      8934
weighted avg       0.93      0.94      0.93      8934


Running NER Inference...

Input: A person was born in Hyderabad and worked at Microsoft.

Predicted Entities:
Entity: hyderabad, Label: LOC, Confidence: 0.9926999807357788
Entity: microsoft, Label: ORG, Confidence: 0.9693999886512756
```

{"model_id":"fb1826c2b17948de8e1e4dfc5a2fcc88","version_major":2,"version_minor":0}

```
============================================================
SELF-SUPERVISED NER PROJECT - DELIVERABLES REPORT
```

```
============================================================
(a) Data & Preprocessing Pipeline: COMPLETED
(b) Self-Supervised Model (MLM): PRE-TRAINED
(c) NER Fine-Tuned Model: TRAINED & SAVED
(d) Evaluation: F1 Score + Classification Report Generated
(e) Deployment: Inference Pipeline Ready
============================================================
```